

FIRMWARE TECHNICAL REFERENCE

ESP32 16-Channel Automatic Relay Timer Switch

Complete Section-by-Section

Function-by-Function

Line-by-Line Documentation

Author: Raff Alds

GitHub: github.com/xiv3r

License: GPLv3

Config Version: EEPROM v11 / EXT_CFG 0xEC

Source: esp32-final-sketch.ino (~3982 lines)

No features skipped · Every API · Every handler · Every structure

This document is a full technical reference for the single-file Arduino firmware that implements a 16-channel automatic relay timer with web UI, NTP/RTC timekeeping, self-healing networking, and NVS persistence. Every section, structure, function, API route, and UI feature is documented.

Table of Contents

- 1. Overview & Feature Summary**
- 2. Includes & External Dependencies**
- 3. Constants, Macros & Configuration Defines**
 - 3.1** NVS / Preferences
 - 3.2** Year 2106+ / 64-bit Time Support
 - 3.3** Day-of-Week Bitmask Constants
 - 3.4** Month-of-Year Bitmask Constants
 - 3.5** Timing Constants
 - 3.6** NTP Async State Machine
 - 3.7** 64-bit NTP Constants
 - 3.8** Memory Management Constants
 - 3.9** Boot Button Factory Reset
 - 3.10** mDNS Settings
 - 3.11** NTP Fallback Pool
 - 3.12** Time Source Tracking
 - 3.13** Default Relay GPIO Pin Map
- 4. Data Structures**
 - 4.1** GPIOPinConfig
 - 4.2** TimerSchedule
 - 4.3** RelayConfig
 - 4.4** SystemConfig
 - 4.5** ExtConfig
 - 4.6** HealthMetrics
 - 4.7** CriticalRelayState
 - 4.8** ResponseCache
- 5. Global Variables**
- 6. SelfHealingSystem Class (Full API)**
- 7. Utility Helpers (Inline)**
- 8. 64-bit Time Conversion Functions**
- 9. GPIO / Relay Output Helpers**
- 10. Schedule Defaults & WiFi Station Control**
- 11. DS3231 RTC Functions**
- 12. Enhanced Internal RTC (Software Clock)**
- 13. Self-Healing Implementation (All Methods)**
- 14. Memory Management Functions**
- 15. Web UI Assets (CSS + 6 HTML Pages)**
- 16. Boot Button Factory Reset**
- 17. GPIO Configuration Persistence**

- 18. NTP Client (64-bit) & Sync**
- 19. Browser Time Sync Handler**
- 20. WiFi Connection Logic**
- 21. Relay Output Engine**
- 22. Schedule Cache & Schedule Engine**
- 23. mDNS Functions**
- 24. setup() — Full Boot Sequence**
- 25. loop() — Main Runtime Loop**
- 26. Configuration Load/Save (NVS)**
- 27. Web Server Route Table**
- 28. REST API Handlers (Every Endpoint)**
- 29. GPIO Management Handlers**
- 30. Captive Portal Endpoints**
- 31. Runtime Architecture & Data Flow**
- 32. Complete Feature Checklist**
- 33. Default Values Reference**
- 34. Error Handling & Edge Cases**
- 35. Hardware Pinout & Wiring Notes**

1. Overview & Feature Summary

This firmware implements a complete **ESP32-based 16-channel automatic relay timer switch** with a built-in web configuration UI, dual WiFi modes (AP + Station), multi-source timekeeping (NTP, browser sync, DS3231 hardware RTC, and software internal RTC with drift compensation), per-relay scheduling (up to 8 schedules per channel with day-of-week, day-of-month, and month-of-year masks), manual override, dynamic GPIO pin assignment, active-low/active-high polarity control, self-healing network services, captive portal support, mDNS discovery, NVS persistence, and a hardware BOOT-button factory reset.

The source file is a single Arduino sketch (~3982 lines) combining C++ firmware logic, PROGMEM-stored CSS/HTML/JS web assets, and a full REST JSON API. Configuration version is **EEPROM_VERSION 11** with extended config magic **0xEC**.

1.1 Complete Feature List (No Skips)

- ✓ Up to 16 independently controlled relay channels (dynamically configurable count)
- ✓ 8 independent timer schedules per relay (start/stop with second precision)
- ✓ Day-of-week bitmask scheduling (Sun–Sat, weekdays, weekends, custom)
- ✓ Day-of-month bitmask scheduling (days 1–31)
- ✓ Month-of-year bitmask scheduling (Jan–Dec) — v11 feature
- ✓ Overnight (cross-midnight) schedule support
- ✓ Always-ON mode when start time equals stop time
- ✓ Manual ON/OFF override with return-to-auto
- ✓ Custom relay names (up to 15 characters, double-click rename in UI)
- ✓ Active-LOW / Active-HIGH polarity per relay or global override
- ✓ Dynamic GPIO pin add/remove/reorder via web UI
- ✓ Default 16-pin map: 32,33,25,26,27,14,13,23,1,3,19,18,5,4,2,15
- ✓ WiFi Station mode (connect to existing network) with enable/disable toggle
- ✓ WiFi Access Point mode (always available for local configuration)
- ✓ AP channel (1–13) and hidden SSID support
- ✓ Async WiFi network scanner with RSSI bars and encryption indicators
- ✓ WiFi reconnect with backoff (max 10 attempts, 5-minute cool-down)
- ✓ WiFi scan pause system (temporarily disconnects STA during scan)
- ✓ 64-bit NTP client with multi-server fallback pool (4 servers)
- ✓ Configurable NTP auto-sync interval (1–24 hours)
- ✓ Browser time sync fallback (POST utc_epoch from client JS)
- ✓ DS3231 hardware RTC on I2C (GPIO 21 SDA / 22 SCL)
- ✓ Software internal RTC with microsecond-based drift compensation
- ✓ RTC rebase every 5 minutes; auto-save internal epoch hourly when offline
- ✓ Time source tracking: NONE / NTP / BROWSER / RTC
- ✓ Year 2106+ support via 64-bit Unix epoch arithmetic
- ✓ Captive portal (DNS wildcard + iOS/Android/Windows detect URLs)
- ✓ mDNS hostname (default from AP SSID, sanitised, .local discovery)
- ✓ Self-healing system: live reconfigure WiFi/mDNS/DNS/Web/AP + smart recovery
- ✓ Critical relay state checkpoint to NVS (magic 0xDEADBEEF + checksum)
- ✓ Memory cleanup (heap checks, scan delete, response cache invalidation)
- ✓ Response JSON cache for relays/system/time
- ✓ Factory reset via web API or BOOT button hold (GPIO 0, 5 seconds)
- ✓ Full web UI: Relays, WiFi, Time/NTP, AP, GPIO, System pages
- ✓ Sticky header with live clock + WiFi/time-source status dots
- ✓ Toast notifications, mobile-responsive CSS
- ✓ NVS persistence for sysConfig, extConfig, gpioConfig, relayConfigs, criticalState

- ✓ Config version migration (v10→v11 monthMask fill)
- ✓ Service verification without reboot (targeted recovery)
- ✓ Schedule engine debounce (500 ms) to avoid relay chatter
- ✓ Schedule process interval 250 ms; cache refresh 1000 ms

2. Includes & External Dependencies

Lines 10–18 pull in all required Arduino-ESP32 and third-party libraries:

Line	Include	Purpose
10	<code>#include <WiFi.h></code>	ESP32 WiFi STA + AP stack
11	<code>#include <WiFiUdp.h></code>	UDP sockets for raw NTP packets
12	<code>#include <WebServer.h></code>	HTTP server on port 80
13	<code>#include <DNSServer.h></code>	Captive portal DNS (port 53)
14	<code>#include <Preferences.h></code>	NVS key-value storage (flash)
15	<code>#include <ArduinoJson.h></code>	JSON parse/serialize for REST API
16	<code>#include <ESPmDNS.h></code>	Multicast DNS (.local hostname)
17	<code>#include <Wire.h></code>	I2C bus for DS3231 RTC
18	<code>#include <RTCLib.h></code>	Adafruit RTCLib — RTC_DS3231 class

Header comment block (lines 1–8) identifies the project title, author Raff Alds, GitHub URL <https://www.github.com/xiv3r>, and GPLv3 license.

3. Constants, Macros & Configuration Defines

3.1 Preferences (NVS) — Lines 20–25

```
L23 Preferences preferences;
    → Global Preferences object for NVS read/write.
L24 #define NVS_NAMESPACE "relay16"
    → Flash namespace key; all prefs stored under this partition name.
L25 #define EEPROM_MAGIC 0x1234
    → Magic number validating SystemConfig blob integrity.
L26 #define EEPROM_VERSION 11
    → Current config schema version; triggers migration if older.
L27 #define EXT_CFG_MAGIC 0xEC
    → Magic byte validating ExtConfig blob.
```

3.2 Year 2106+ / 64-bit Time Support — Lines 29–33

Standard 32-bit Unix time overflows in January 2038. This firmware uses `uint64_t` epochs throughout and validates ranges with `VALID_UNIX_TIME_64`.

```
L31 #define MAX_UNIX_TIME_64 18446744073709551615ULL
    → Maximum uint64_t value (sentinel upper bound).
L32 #define MIN_UNIX_TIME_64 1000000000ULL
    → Minimum accepted epoch (~2001-09-09); rejects unset/zero clocks.
L33 #define VALID_UNIX_TIME_64(epoch) (...)
    → Macro: true if epoch is strictly between MIN and MAX.
```

3.3 Day-of-Week Bitmask Constants — Lines 35–45

Each day is a single bit in a `uint8_t`. Bit 0 = Sunday ... Bit 6 = Saturday. Schedules store a `days[s]` mask; a schedule fires only when `(days & todayBit) != 0`.

Constant	Value	Meaning
<code>DAY_SUNDAY</code>	<code>1 << 0 = 0x01</code>	Sunday
<code>DAY_MONDAY</code>	<code>1 << 1 = 0x02</code>	Monday
<code>DAY_TUESDAY</code>	<code>1 << 2 = 0x04</code>	Tuesday
<code>DAY_WEDNESDAY</code>	<code>1 << 3 = 0x08</code>	Wednesday

Constant	Value	Meaning
DAY_THURSDAY	1 << 4 = 0x10	Thursday
DAY_FRIDAY	1 << 5 = 0x20	Friday
DAY_SATURDAY	1 << 6 = 0x40	Saturday
DAY_ALL	0x7F	All seven days (Everyday)
DAY_WEEKDAYS	0x3E	Mon–Fri (bits 1–5)
DAY_WEEKENDS	0x41	Sat+Sun (bits 0 and 6)

3.4 Month-of-Year Bitmask Constants — Lines 47–61

Introduced in config v11. Each month is a bit in a uint16_t monthMask. MONTH_ALL = 0x0FFF selects every month. Bit 0 = January ... Bit 11 = December.

Constant	Bit	Month
MONTH_JANUARY	1<<0	January
MONTH_FEBRUARY	1<<1	February
MONTH_MARCH	1<<2	March
MONTH_APRIL	1<<3	April
MONTH_MAY	1<<4	May
MONTH_JUNE	1<<5	June
MONTH_JULY	1<<6	July
MONTH_AUGUST	1<<7	August
MONTH_SEPTEMBER	1<<8	September
MONTH_OCTOBER	1<<9	October
MONTH_NOVEMBER	1<<10	November
MONTH_DECEMBER	1<<11	December
MONTH_ALL	0x0FFF	All 12 months

3.5 Timing Constants — Lines 63–73

Constant	Value (ms)	Role
NTP_RETRY_INTERVAL	30 000	Minimum gap between NTP attempts
WIFI_CHECK_INTERVAL	10 000	How often loop checks WiFi status
WIFI_CONNECT_TIMEOUT	20 000	Give up a single connect attempt after 20s
RTC_UPDATE_INTERVAL	100	Reserved RTC tick (legacy)
SCHEDULE_PROCESS_INTERVAL	250	Run schedule engine 4× per second
RELAY_UPDATE_INTERVAL	500	Relay output debounce baseline
RTC_REBASE_INTERVAL	300 000	Rebase software RTC every 5 minutes
RTC_SYNC_INTERVAL	3 600 000	Legacy 1-hour RTC sync interval
DS3231_SYNC_INTERVAL	3 600 000	Pull time FROM DS3231 every hour

3.6 NTP Async State Machine — Lines 75–80

```

L77 NTP_SERVER_TIMEOUT = 5000UL
    → Per-server wait timeout (5 s).
L78 NTP_STATE_IDLE = 0
    → No NTP operation in progress.
L79 NTP_STATE_CONNECTING = 1
    → Opening UDP / sending request.
L80 NTP_STATE_WAITING = 2
    → Awaiting response packet.
```


3.7 64-bit NTP Constants — Lines 82–88

Constant	Value	Meaning
NTP_PACKET_SIZE	48	Standard NTP packet length in bytes
NTP_PORT	123	IANA NTP UDP port
NTP_TIMEOUT	5000	ms to wait for a reply
NTP_RETRY_COUNT	3	Retries per server inside getNTP64Time
NTP_EPOCH_OFFSET	2208988800ULL	Seconds between 1900-01-01 and 1970-01-01

3.8 Memory Management Constants — Lines 90–94

L92 MEMORY_CLEANUP_INTERVAL = 30000UL
→ Periodic stale-resource cleanup every 30 s.

L93 MEMORY_CHECK_INTERVAL = 60000UL
→ Declared interval for memory checks.

L94 CONNECTION_TIMEOUT = 10000UL
→ Stale HTTP connection timeout helper.

3.9 Boot Button Factory Reset — Lines 96–101

L98 BOOT_BUTTON_PIN = 0
→ ESP32 BOOT (GPIO0). Active LOW when pressed.

L99 FACTORY_RESET_HOLD = 5000
→ Must hold 5 continuous seconds to trigger reset.

L100 bootButtonPressStart = 0
→ millis() timestamp when press began.

L101 bootButtonPressed / factoryResetTriggered
→ Edge-detect and one-shot guard flags.

3.10 mDNS Settings — Lines 103–105

L104 MDNS_HOSTNAME_DEFAULT "esp32"
→ Fallback hostname if AP SSID sanitises to empty.

L105 MDNS_RESTART_DELAY = 2000UL
→ Delay before applying a hostname change (2 s).

3.11 NTP Fallback Pool — Lines 107–114

Four public NTP servers rotated on failure:

- NTP_SERVERS[0] = "time.google.com"
- NTP_SERVERS[1] = "time.windows.com"
- NTP_SERVERS[2] = "time.cloudflare.com"
- NTP_SERVERS[3] = "time.facebook.com"

L114 NUM_NTP_SERVERS = 4
→ Pool size used by tryNTPSync / recoverNTP.

3.12 Time Source Tracking — Lines 116–122

Constant	Value	Meaning
TIME_SOURCE_NONE	0	Clock not yet set
TIME_SOURCE_NTP	1	Last good sync from NTP
TIME_SOURCE_BROWSER	2	Last good sync from browser POST
TIME_SOURCE_RTC	3	Running from DS3231 / internal RTC

L121 static uint8_t timeSource = TIME_SOURCE_NONE
→ Current authoritative time origin.

L122 static unsigned long lastBrowserSync = 0
→ millis of last successful browser sync.

3.13 DS3231 RTC Objects — Lines 124–129

L126 `RTC_DS3231 rtc;`
 → Hardware RTC driver instance.

L127 `bool rtcPresent = false;`
 → True after successful `rtc.begin()`.

L128 `unsigned long lastRTCDSync = 0;`
 → millis of last DS3231 read/write.

L129 `bool rtcTimeValid = false;`
 → True if DS3231 holds a plausible date (2020–2100).

3.14 WiFi Scan Pause System — Lines 131–136

When a scan is requested while STA is still connecting, the firmware disconnects STA for `WIFI_PAUSE_DURATION` (15 s) so the radio can complete a clean scan, then resumes reconnect.

L133 `wifiPausedForScan`
 → True while STA is intentionally paused.

L134 `wifiPauseUntil`
 → millis deadline to resume STA.

L135 `WIFI_PAUSE_DURATION = 15000UL`
 → 15-second pause window.

L136 `lastScanAttempt`
 → millis of last pause-for-scan action.

3.15 Default Relay GPIO Pin Map — Lines 168–187

`DEFAULT_RELAY_PINS[]` maps logical relay index → ESP32 GPIO. Comment in source says “change gpio pins” — these are board-specific and match the GPIO page `DEFAULT_PINS` JS array.

Relay	GPIO	Label	Relay	GPIO	Label
1	32	IN1	9	1	IN9
2	33	IN2	10	3	IN10
3	25	IN3	11	19	IN11
4	26	IN4	12	18	IN12
5	27	IN5	13	5	IN13
6	14	IN6	14	4	IN14
7	13	IN7	15	2	IN15
8	23	IN8	16	15	IN16

Note: GPIO 1 (TX0) and GPIO 3 (RX0) are UART0 pins — using them as relays disables USB-serial logging after `pinMode OUTPUT`. GPIO 2 is often tied to onboard LED.

4. Data Structures

4.1 GPIOPinConfig — Lines 192–198

```
struct GPIOPinConfig {
    uint8_t pins[MAX_RELAYS];    // GPIO number per relay slot
    uint8_t count;               // Active relay count (0..16)
    uint16_t magic;              // Must equal GPIO_CONFIG_MAGIC 0xD002
    bool activeLow[MAX_RELAYS];  // true = energise on LOW level
};
```

Persisted under NVS key "gpioConfig". If magic/count invalid on load, defaults to all 16 DEFAULT_RELAY_PINS with activeLow=true for every channel.

4.2 TimerSchedule — Lines 205–213

```
struct TimerSchedule {
    uint8_t startHour[8], startMinute[8], startSecond[8];
    uint8_t stopHour[8], stopMinute[8], stopSecond[8];
    bool enabled[8];           // schedule slot active?
    uint8_t days[8];           // DOW bitmask (DAY_*)
    uint32_t monthDays[8];     // DOM bitmask bits0-30 = days1-31; 0=any day
    uint16_t monthMask[8];     // MOY bitmask (MONTH_*); 0 or MONTH_ALL = all
};
```

Eight parallel slots. A zero monthDays means “any day of month”. A zero or MONTH_ALL monthMask means “any month”. Day-of-week zero means no days (never fires) unless combined carefully — defaults initialise days[] to DAY_ALL.

4.3 RelayConfig — Lines 216–221

```
struct RelayConfig {
    TimerSchedule schedule;    // 8 timer slots
    bool manualOverride;       // true → ignore schedules
    bool manualState;          // desired state when overridden
    char name[16];             // display name (15 chars + NUL)
};
```

4.4 SystemConfig — Lines 224–237 (packed)

```
struct SystemConfig {
    uint16_t magic;            // EEPROM_MAGIC 0x1234
    uint8_t version;           // EEPROM_VERSION 11
    char sta_ssid[32];         // WiFi station SSID
    char sta_password[64];     // WiFi station password
    char ap_ssid[32];          // SoftAP SSID
    char ap_password[32];     // SoftAP password (blank = open)
    char ntp_server[48];       // Preferred NTP hostname
    int32_t gmt_offset;         // seconds east of UTC (default 28800 = UTC+8)
    int32_t daylight_offset;    // DST seconds (default 0)
    uint64_t last_rtc_epoch;    // persisted software clock
    float rtc_drift;           // drift compensation factor
    char hostname[32];         // mDNS / device hostname seed
} __attribute__((packed));
```

Packed to guarantee stable sizeof() across NVS getBytes/putBytes.

4.5 ExtConfig — Lines 240–248 (packed)

```
struct ExtConfig {
    uint8_t magic;             // EXT_CFG_MAGIC 0xEC
    uint8_t ap_channel;        // 1..13 (default 6)
    uint8_t ntp_sync_hours;    // 1..24 (default 1)
    uint8_t ap_hidden;         // 0=broadcast, 1=hidden
    uint8_t global_active_mode; // 0=per-relay, 1=all LOW, 2=all HIGH
    uint8_t sta_enabled;       // 0=AP-only, 1=STA allowed
    uint8_t reserved[26];      // future expansion, zeroed
};
```

```
} __attribute__((packed));
```

4.6 HealthMetrics — Lines 253–261

```
struct HealthMetrics {  
    uint32_t wifiFailures = 0;  
    uint32_t ntpFailures = 0;  
    uint32_t mdnsFailures = 0;  
    uint32_t dnsFailures = 0;  
    uint32_t webServerFailures = 0;  
    unsigned long lastRecoveryAttempt = 0;  
    bool inRecoveryMode = false;  
};
```

Used by SelfHealingSystem::smartRecovery to throttle and count failures.

4.7 CriticalRelayState — Lines 262–268

```
struct CriticalRelayState {  
    uint32_t magic = 0xDEADBEEF;  
    bool relayStates[MAX_RELAYS];  
    bool manualOverrides[MAX_RELAYS];  
    uint32_t timestamp;    // millis snapshot  
    uint32_t checksum;    // XOR integrity  
};
```

Saved to NVS key "criticalState". On boot, restoreCriticalState() re-applies only relays that were under manual override so manual states survive soft resets.

4.8 ResponseCache — Lines 355–361

```
struct ResponseCache {  
    String relaysJson;  
    String systemJson;  
    String timeJson;  
    unsigned long lastUpdate = 0;  
    bool valid = false;  
};
```

Intended to reduce heap churn on frequent /api/relays polls. handleGetRelays checks a 2-second freshness window; cleanupStaleResources invalidates after 5 seconds.

5. Global Variables

5.1 Configuration Instances — Lines 273–279

Variable	Type	Role
sysConfig	SystemConfig	Main persistent system settings
extConfig	ExtConfig	Extended AP/NTP/polarity/STA flags
relayConfigs[MAX_RELAYS]	RelayConfig[16]	Per-channel schedules + manual + name
health	HealthMetrics	Self-healing counters
criticalState	CriticalRelayState	Checkpoint blob
criticalStateInitialized	bool	True after first successful save
criticalStateDirty	bool	Needs flush to NVS

5.2 Software RTC State — Lines 281–291

Variable	Role
internalEpoch	uint64_t base UTC seconds at last rebase/sync
internalMillisAtLastSync	millis() at last external sync
driftCompensation	float multiplier on elapsed real time (default 1.0)
rtcInitialized	Software clock has a valid base epoch
lastRTCUpdate	millis of last update touch
rtcMicrosAtLastSync	micros() anchor for high-res elapsed calc
lastRTCRebase	millis of last performRTCRebase()
lastInternalRTCSave	millis of last auto-save to NVS
INTERNAL_RTC_SAVE_INTERVAL	3 600 000 ms (1 hour)

5.3 NTP Runtime — Lines 293–301

Variable	Role
ntpServerIndex	Current index into NTP_SERVERS[]
ntpFailCount	Consecutive failure counter
lastNTPSync	millis of last successful NTP apply
lastNTPAttempt	millis of last tryNTPSync call
ntpAsyncState	IDLE / CONNECTING / WAITING
ntpAsyncCurrentServer	Server under async attempt
ntpAsyncPhaseStart	Phase start millis

5.4 WiFi Runtime — Lines 303–320

Variable	Role
wifiConnected	Logical STA connected flag
lastWiFiCheck	Last status poll millis
wifiReconnectAttempts	Attempts since last success
wifiGiveUpUntil	Backoff deadline after MAX_RECONNECT
MAX_RECONNECT = 10	Attempts before 5-minute cool-down
wifiConnecting	begin() issued, awaiting WL_CONNECTED
wifiConnectStart	millis when begin() called

Variable	Role
wifiFirstAttempt	Skips cool-down on first boot attempt
scanInProgress	Async scan outstanding
scanResultCount	Last scan result count (-1 = none)
scanStartTime	Scan timeout anchor (10 s)
ap_ssid / ap_password	Runtime AP credential copies

5.5 mDNS Runtime — Lines 326–330

- mdnsStarted — true after MDNS.begin success
- mdnsHostname[32] — current hostname string
- mdnsRestartPending / mdnsRestartScheduled — deferred restart after rename

5.6 Schedule Engine Cache — Lines 335–345

Variable	Role
cachedTodayBit	1 << tm_wday for local today
cachedMonthDay	tm_mday (1–31)
cachedMonth	tm_mon (0–11)
lastScheduleEpoch	UTC epoch when cache was built
scheduleActiveCache[16]	Desired ON/OFF from schedules only
lastScheduleCacheUpdate	millis of last cache rebuild
SCHEDULE_CACHE_INTERVAL	1000 ms
lastScheduleProcess	millis of last processRelaySchedules
lastRelayUpdate	millis of last bulk output update
lastRelayOutputs[16]	Last driven logical states
relayOutputsInitialized	First drive completed

5.7 Memory / Connection Tracking — Lines 350–354

- lastMemoryCleanup, lastHeapCheck, minFreeHeap, lastConnectionActivity, lastServerRestart

6. SelfHealingSystem Class Declaration — Lines 366–390

Class aggregates all live-reconfigure and recovery operations so the main loop can call a single `smartRecovery()` tick. Instance name: **healer** (line 395).

```
class SelfHealingSystem {
public:
    bool liveReconfigureWiFi();
    bool liveReconfigureMDNS();
    bool liveReconfigureDNS();
    bool liveReconfigureWebServer();
    bool liveReconfigureAP();
    bool recoverWiFi();
    bool recoverMDNS();
    bool recoverDNS();
    bool recoverWebServer();
    bool recoverNTP();
    bool recoverRTC();
    void smartRecovery();
    void verifyRelayStates();
    void saveCriticalState();
    bool restoreCriticalState();
    void performTargetedRecovery();
    void restartAPIIfNeeded(bool forceRestart = false);
private:
    unsigned long lastMDNSAnnounce;
    unsigned long lastWebServerCheck;
    unsigned long lastFullHealthCheck;
    unsigned long lastWiFiReconfigure;
    unsigned long lastMDNSReconfigure;
    unsigned long lastDNSReconfigure;
};
```

Forward declarations (lines 396–455) list every free function defined later so the compiler accepts calls before definitions (Arduino single-translation-unit style).

7. Utility Helpers (Inline)

■ inline bool timeHasElapsed(current, previous, interval) — Lines 141–143

Millis-safe elapsed check using unsigned subtraction (handles millis() wrap every ~49.7 days).

current (unsigned long) — Usually millis()
previous (unsigned long) — Timestamp of last event
interval (unsigned long) — Required gap in ms

Returns: true if (current – previous) ≥ interval.

```
L142 return (current - previous) >= interval;
→ Unsigned math makes wrap-around correct.
```

■ inline bool isTimeReached(current, target) — Lines 148–150

True when current has reached or passed a future deadline stored as absolute millis.

```
L149 return (long)(current - target) >= 0;
→ Cast difference to signed long for deadline compare.
```

■ inline uint64_t rtcDateTimeToUint64(DateTime dt) — Lines 460–462

```
L461 return (uint64_t)dt.unixtime();
→ Widen RTCLib 32-bit unixtime to 64-bit.
```

■ inline DateTime uint64ToRtcDateTime(uint64_t t64) — Lines 463–465

```
L464 return DateTime((uint32_t)(t64 & 0xFFFFFFFFFULL));
→ Truncate to low 32 bits for DS3231 (valid through 2106).
```

8. 64-bit Time Conversion Functions

■ `struct tm* gmtime64(uint64_t* timep)` — Lines 470–520

Custom UTC broken-down time conversion supporting full `uint64_t` epochs (beyond 2038). Uses the Howard Hinnant `civil_from_days` algorithm (era/doe/yoe/doy/mp).

timep (`uint64_t*`) — Pointer to UTC Unix seconds

Returns: pointer to static struct `tm` (not thread-safe; single-core OK).

Line-by-line algorithm

```
L471 static struct tm tm;
    → Reusable static buffer.
L472 uint64_t t = *timep;
    → Dereference epoch.
L-11 if (t == 0) { ... epoch zero → 1970-01-01 Thu }
    → Special-case empty clock: returns Unix epoch calendar fields (tm_wday=4 Thursday).
L485 uint64_t days = t / 86400ULL;
    → Whole days since 1970-01-01.
L486 uint64_t rem = t % 86400ULL;
    → Seconds within the day.
L-3 tm_sec / tm_min / tm_hour from rem
    → Break remainder into h:m:s.
L491 uint64_t z = days + 719468ULL;
    → Shift to civil algorithm origin (0000-03-01 based).
L492 era = z / 146097
    → 400-year Gregorian eras.
L493 doe = z - era * 146097
    → Day of era [0, 146096].
L494 yoe = (doe - doe/1460 + doe/36524 - doe/146096) / 365
    → Year of era [0, 399].
L495 y = yoe + era * 400
    → Absolute year.
L496 doy = doe - (365*yoe + yoe/4 - yoe/100)
    → Day of year.
L-3 mp, d, m month computation
    → Convert day-of-year to month/day; adjust year if month ≤ 2 (Jan/Feb).
L-3 tm_mday, tm_mon, tm_year, tm_wday
    → Fill POSIX tm (tm_year = years since 1900; tm_wday = (days+4)%7).
L-2 tm_yday, tm_isdst=0, return &tm
    → Finish and return.
```

■ `struct tm* localtime64(uint64_t* timep)` — Lines 525–528

```
L526 localTime = *timep + gmt_offset + daylight_offset
    → Apply timezone then reuse gmtime64.
L527 return gmtime64(&localTime);
    → Broken-down local time.
```

■ `inline uint64_t getLocalEpoch(uint64_t utcEpoch)` — Lines 533–535

```
L534 return utcEpoch + (uint64_t)(gmt_offset + daylight_offset);
    → Local epoch seconds without building struct tm.
```

9. GPIO / Relay Output Helpers

■ `inline bool isActiveLow(uint8_t index)` — Lines 540–550

Resolves effective polarity for relay index with global mode override.

```
L541 if (index < gpioConfig.count)
    → Bounds check.
```



```
L-1 global_active_mode == 1 → return true
  → Force all active-LOW.
L-1 global_active_mode == 2 → return false
  → Force all active-HIGH.
L546 return gpioConfig.activeLow[index];
  → Per-relay setting when mode==0.
L548 return true;
  → Out-of-range default: active-LOW (safe fail).
```

■ inline unsigned long getNTPInterval() — Lines 555–559

```
L-1 clamp ntp_sync_hours to 1..24
  → Invalid values fall back to 1 hour.
L558 return h * 3600000UL;
  → Convert hours → milliseconds.
```

■ inline int getRelayPin(uint8_t index) — Lines 560–565

```
L-2 return gpioConfig.pins[index] if in range else -1
  → Negative signals unused slot.
```

■ inline uint8_t getActiveRelayCount() — Lines 566–568

```
L567 return gpioConfig.count;
  → Current configured channel count.
```

■ inline void setRelayOutput(uint8_t index, bool state) — Lines 573–578

Drives the physical pin. **state=true** means relay energised (logical ON). If active-low, the GPIO is written inverted.

```
L574 int pin = getRelayPin(index);
  → Resolve GPIO number.
L-2 digitalWrite(pin, isActiveLow ? !state : state);
  → Apply polarity inversion when needed.
```

10. Schedule Defaults & WiFi Station Control

■ void initScheduleDefaults(int relayIndex) — Lines 583–592

Zeroes a RelayConfig and applies factory schedule/name defaults for one channel.

```
L584 memset(&relayConfigs[relayIndex], 0, sizeof(RelayConfig));
    → Clear all fields including manual flags and times.
L-4 for s in 0..7: days=DAY_ALL, monthDays=0, monthMask=MONTH_ALL
    → Every schedule slot: all weekdays, any DOM, all months; enabled remains false (zeroed).
L590 sprintf(name, 16, "Relay %d", relayIndex+1);
    → Human label "Relay 1" ... "Relay 16".
```

■ void setWiFiStationEnabled(bool enabled) — Lines 597–627

Runtime enable/disable of STA mode without reboot. Persists to ExtConfig.

```
L-1 extConfig.sta_enabled = enabled?1:0; saveExtConfig();
    → Persist flag immediately.
```

When disabling (lines 600–611)

```
L601 WiFi.disconnect(true);
    → Drop STA association and erase credentials from driver.
L-4 clear wifiConnected/Connecting/Paused/attempts/giveUp
    → Reset all STA state machines.
L-2 WiFi.mode(WIFI_AP);
    → AP-only radio mode.
L-1 ntpAsyncState=IDLE; lastNTPSync=0;
    → Stop NTP activity (no internet path).
```

When enabling (lines 612–624)

```
L-2 WiFi.mode(WIFI_AP_STA);
    → Dual mode.
L-7 if sta_ssid set → WiFi.begin(...); mark connecting
    → Kick off association with stored credentials.
L625 responseCache.valid = false;
    → Force UI refresh of connection status.
```

11. DS3231 RTC Functions

■ void initRTC() — Lines 632–659

Called at the very start of setup(). Probes I2C DS3231 and optionally seeds software clock.

```
L633 Wire.begin(21, 22);
    → I2C on default ESP32 pins: SDA=21, SCL=22.
L-4 if (!rtc.begin()) { present=false; return; }
    → No ACK on I2C → hardware absent; continue without RTC.
L639 rtcPresent = true;
    → Chip responded.
L-3 if (rtc.lostPower()) { rtcTimeValid=false; return; }
    → Oscillator stop / battery fail flag — do not trust time.
L-13 read now; if year 2020..2100 and VALID → seed internalEpoch
    → Copy hardware time into software RTC; set timeSource=RTC; drift=1.0; anchors micros/millis.
```

■ void immediateDS3231Sync() — Lines 661–670

Pushes current internalEpoch OUT to the DS3231 (write direction).

```
L662 guard: rtcPresent && rtcInitialized && internalEpoch>0
    → Only write if we have something meaningful.
```

```
L-5 rtc.begin(); rtc.adjust(uint64ToRtcDateTime(internalEpoch));  
→ Re-init bus, convert epoch→DateTime, program chip; stamp lastRTCDSync.
```

■ void syncDS3231FromInternalRTC() — Lines 672–680

Same write direction as immediateDS3231Sync but without re-calling rtc.begin(); used by recovery paths.

```
L-1 early return if !present or !initialized  
→ Safety.  
L-4 rtc.adjust(...); rtcTimeValid=true; lastRTCDSync=millis();  
→ Program and mark valid.
```

12. Enhanced Internal RTC (Software Clock)

■ void performRTCRebase() — Lines 685–710 [note: spelling Reabase]

Advances internalEpoch by the high-resolution elapsed time since last anchor, applying driftCompensation. Handles micros() 32-bit wrap. Called on a 5-minute cadence and before saves.

```
L-1 capture currentMicros / currentMillis
  → Fresh anchors.
L-9 compute elapsedMicros with wrap handling
  → If current < anchor, elapsed = (0xFFFFFFFF-anchor)+current+1.
L-2 elapsedSeconds = elapsedMicros/1e6; adjusted = elapsed * drift
  → Scale by driftCompensation (1.0 = perfect crystal).
L-5 add whole seconds; round up if fractional ≥ 0.5
  → Nearest-second quantisation.
L-2 reset rtcMicrosAtLastSync, lastRTCRebase, lastRTCUpdate
  → New baseline after absorbing elapsed into epoch.
```

■ uint64_t getCurrentEpoch() — Lines 711–745

Primary time read used by schedules, API, and NTP. Never blocks.

```
L-2 if software RTC live → maybe rebase if 5 min elapsed
  → Keep base fresh so float elapsed stays small/accurate.
L-9 compute elapsed since last anchor; if micros wrapped → rebase & return base
  → Wrap during read triggers immediate rebase for consistency.
L-8 result = internalEpoch + adjustedSeconds (rounded)
  → Return live extrapolated UTC epoch without mutating base (except via rebase).
L-8 fallback: read DS3231 hardware if software not init
  → Direct chip read when internal clock never seeded.
L744 return 0;
  → No valid time source available.
```

■ void saveRTCState() — Lines 747–752

```
L748 performRTCRebase();
  → Absorb pending elapsed first.
L-2 sysConfig.last_rtc_epoch=internalEpoch; rtc_drift=1.0; saveConfiguration();
  → Persist epoch; note drift forced to 1.0 on this path.
```

■ void loadRTCState() — Lines 754–762

```
L-6 if VALID last_rtc_epoch → restore internalEpoch, drift=1.0, anchors, rtcInitialized=true
  → Boot recovery when DS3231 absent or invalid.
```

■ void autoSaveInternalRTC() — Lines 767–782

Hourly NVS save of software clock — but only when neither NTP nor valid DS3231 is available. Avoids unnecessary flash wear when a better authority exists.

```
L769 return if !rtcInitialized or epoch==0
  → Nothing to save.
L-4 if NTP or DS3231 available → refresh lastInternalRTCsave and return
  → Skip write; external authority is enough.
L-6 every INTERNAL_RTC_SAVE_INTERVAL → rebase + put last_rtc_epoch + drift
  → Offline persistence path.
```

■ bool loadRTCFromDS3231() — Lines 787–804

Explicit pull FROM hardware INTO software clock. Returns true on success.

```
L-14 if present+valid year range+VALID epoch → seed all anchors, timeSource=RTC, return true
  → Full software clock initialisation from chip.
L803 return false;
  → Hardware unusable.
```

■ void pauseWiFiForScan() — Lines 809–817

```
L810 guard: connecting && !connected && !already paused && sta_enabled  
→ Only interrupt in-flight connects.  
L-5 set pause flags; WiFi.disconnect(true); delay(100); clear connecting  
→ Free radio for scanNetworks().
```

13. Self-Healing Implementation (All Methods)

■ `uint32_t calculateCriticalChecksum()` — Lines 822–829

Computes integrity word over relayStates + manualOverrides + timestamp.

- L-3 sum bits for each state/override into shifted positions
→ Lightweight fingerprint, not cryptographic.
- L828 return sum ^ timestamp;
→ Mix in timestamp.

■ `void SelfHealingSystem::saveCriticalState()` — Lines 831–847

- L-3 copy lastRelayOutputs + manualOverride for active relays
→ Snapshot live outputs.
- L-3 zero unused slots beyond count
→ Deterministic blob.
- L-2 timestamp=millis(); checksum=...; magic=0xDEADBEEF
→ Seal structure.
- L-3 preferences.putBytes("criticalState", ...)
→ Write full struct to NVS; mark initialized.

■ `bool SelfHealingSystem::restoreCriticalState()` — Lines 849–867

- L-2 getBytes criticalState; check len+magic
→ Validate blob size and magic.
- L-11 if checksum matches → for each manual override, drive pin + restore flags
→ Only manual channels restored (schedule channels recomputed).
- L-1 return true/false
→ Success indicator.

■ `bool liveReconfigureWiFi()` — Lines 872–891

- L874 throttle: once per 30 s
→ Avoid reconnect storms.
- L875 return if STA disabled
→ No-op in AP-only mode.
- L-12 if SSID set and not connected (or wrong SSID) → WiFi.begin
→ Soft reconnect / SSID mismatch correction.
- L-1 stamp lastWiFiReconfigure; return true
→ Always reports success of attempt.

■ `bool liveReconfigureMDNS()` — Lines 893–912

- L895 throttle: 60 s
→ Rate limit.
- L-8 if not started → MDNS.begin + addService http/tcp/80 + TXT records
→ TXT: model=ESP32, version=v11, channels=N, features=monthmask,64bit.
- L-4 else re-announce service + update TXT
→ Refresh advertisements.

■ `bool liveReconfigureDNS()` — Lines 914–919

Currently a throttled no-op stub (returns true). DNSServer stays running from setup.

■ `bool liveReconfigureWebServer()` — Lines 921–926

Throttled heartbeat stub (30 s). Actual server restart is not performed here; health is observed via lastConnectionActivity elsewhere.

■ `bool liveReconfigureAP()` — Lines 928–943

- L929 if softAPIP == "0.0.0.0"
→ AP interface down → restart SoftAP.
- L-9 clamp channel 1..13; apply hidden flag; softAP(ssid, pass|NULL)
→ Recreate AP with current config; then reconfigure DNS+mDNS.

■ void restartAPIfNeeded(bool forceRestart=false) — Lines 945–968

L-21 if force: softAPdisconnect, delay 500, ensure mode AP/AP_STA, softAP, delay, DNS+mDNS
→ Hard AP bounce used after AP settings change or factory reset.

■ bool recoverWiFi() — Lines 973–986

Similar to liveReconfigureWiFi but returns false when throttled; true if begin issued or already connected.

■ bool recoverMDNS/DNS/WebServer() — Lines 988–996

Thin wrappers delegating to corresponding liveReconfigure* methods.

■ bool recoverNTP() — Lines 998–1015

L999 require wifiConnected && sta_enabled
→ No WAN → fail.
L-12 rotate through all NUM_NTP_SERVERS calling getNTP64Time
→ On first epoch > 1577836800 (2020-01-01), syncInternalRTC and return true.
L1013 delay(100); yield();
→ Breathing room between servers.

■ bool recoverRTC() — Lines 1018–1036

L-1 require rtcPresent; Wire.begin(21,22); rtc.begin()
→ Re-init I2C path.
L-5 if internalEpoch>0 → push to DS3231
→ Software → hardware.
L-6 else if rtcTimeValid → pull hardware into internalEpoch
→ Hardware → software.

■ void performTargetedRecovery() — Lines 1038–1055

Ordered full-stack recovery used by /api/reset and factory reset:

- liveReconfigureWebServer → DNS → mDNS → WiFi → AP (50 ms gaps)
- recoverRTC → verifyRelayStates
- zero health failure counters

■ void verifyRelayStates() — Lines 1057–1075

Every 30 s, read each GPIO and compare to expected logical level (accounting for active-low). Mismatch → re-drive with setRelayOutput. Detects external glitches or pin mode loss.

L-3 expected = manual ? manualState : scheduleActiveCache
→ Authoritative desired logical state.
L-6 expected pin level vs digitalRead; correct if differ
→ Hardware reconciliation.

■ void smartRecovery() — Lines 1077–1110

Called from loop every 10 s. Multi-part health tick:

L-1 throttle 10 s via health.lastRecoveryAttempt
→ Global rate limit.
L-9 if STA enabled, not connected, not connecting, not paused → ++wifiFailures
→ At ≥3 failures, force WiFi.begin and reset counter.
L-3 if connected → wifiConnected=true; clear failures
→ Sync logical flag with driver.
L-4 every 5 min re-add mDNS http service
→ Keep advertisements alive.
L-7 every 30 min full check: mDNS announce + flush dirty critical state
→ Long-period maintenance.
L-1 verifyRelayStates(); liveReconfigureAP();
→ Always verify pins and ensure AP IP is non-zero.

14. Memory Management Functions

■ void performMemoryCleanup() — Lines 1112–1127

```
L-2 WiFi.scanDelete() if not scanning
  → Free scan result buffers.
L-3 yield+delay loop ×10
  → Let IDLE task run / coalesce heap.
L-1 preferences.end(); delay(50);
  → Ensure NVS handle closed.
L-4 malloc(512)/free ×5
  → Touch allocator to reduce fragmentation.
```

■ void cleanupStaleResources() — Lines 1129–1141

```
L-2 abort orphan WIFI_SCAN_RUNNING
  → Defensive scan cleanup.
L-5 if cache older than 5 s → invalidate and clear Strings
  → Release JSON string heap.
L1140 performMemoryCleanup();
  → Always finish with deep clean.
```

■ void checkWebServerHealth() — Lines 1143–1149

```
L-3 if no client activity for 300 s → liveReconfigureWebServer
  → Soft touch after 5 minutes idle.
```

■ void checkAndCleanMemory() — Lines 1151–1166

```
L-8 every 300 s: track minFreeHeap; if freeHeap &lt; 20 KB → cleanup
  → Low-memory emergency path.
L-3 every 3600 s: unconditional performMemoryCleanup
  → Hourly hygiene.
```

15. Web UI Assets (CSS + 6 HTML Pages)

All UI is stored in flash via PROGMEM raw string literals (R"css(...)css" / R"raw(...)raw"). Served by WebServer send_P to avoid RAM copies of large pages.

15.1 style_css[] — SHARED CSS (starts ~line 1171)

Complete design system used by every page:

- Reset: box-sizing border-box; zero margin/padding on *
- Body: system font stack, #EEF2F7 background, 14px base
- Sticky gradient header (#1565C0→#0D47A1) with flex nav + status dots
- Nav link hover/current states with translucent white pills
- Status dots: .g green, .r red, .y yellow, .b blue
- Card grid auto-fill minmax(340px); hover elevation
- Badges: .bon ON green, .boff OFF red, .bman MANUAL orange
- Buttons: ON/OFF/Auto/Save/Sync/Danger/Warn/Scan variants
- Schedule list with scroll; day/mday/month toggle chips
- Night/always-on badges for overnight schedules
- Form controls, alerts (.aw warn, .ai info), network scan list
- RSSI bar component (.bars/.bar)
- Toast #toast with slide-up animation; .ok/.er colours
- Mobile @media max-width 500px: single column, smaller chips

15.2 index_html[] — Relays Page

Main control surface. Features implemented in embedded JavaScript:

JS Function	Behaviour
toast(m,ok)	Show bottom toast 3 s (green/red)
tick()	Poll /api/time every 1 s; update clock + dots
escapeHtml(text)	XSS-safe text via textContent→innerHTML
load()	GET /api/relays → render()
toTS / fromTS	h,m,s ↔ HH:MM:SS for <input type=time step=1>
dayMaskToStr	0x7F→Everyday else list day names
monthDayMaskToStr	Bitmask → comma day numbers
monthMaskToStr	0x0FFF→All months else month names
nightBadge(sc)	Always ON / Overnight / info badge HTML
startEditName	Double-click name → inline input maxLength 15
cancelEdit / saveNameEdit	Escape cancels; Enter/blur POSTs /api/relay/name
render()	Build card per relay: badges, ON/OFF/Auto, 8 schedule editors
toggleDay/MonthDay/Month	XOR bit in local model + refresh chip UI
uf(ri,si,field,val)	Update enabled/start/stop fields live
mc(ri,state)	POST /api/relay/manual {relay,state}
ra(ri)	POST /api/relay/reset → return to Auto
save(ri)	POST /api/relay/save {relay, schedules[8]}

Each schedule editor exposes: enable checkbox, start/stop time (seconds precision), 7 day chips, 31 month-day chips, 12 month chips. Save persists only that relay's schedules.

15.3 wifi_html[] — WiFi Station Page

JS Function	Behaviour
updateStaButton	Style ON (blue) / OFF (red) toggle
loadWiFiStatus	GET /api/wifi → SSID, IP, RSSI, sta_enabled
rsiBar(rssi)	4-segment signal strength graphic
toggleStation	POST {sta_enabled} flip; dim config when off
startScan / pollScan / endScan	POST scan, poll every 2.5 s until done
renderNets	Sort by RSSI; click fills SSID field
saveWiFi	POST {ssid,password}; redirect home after 3 s

When STA disabled, device stays reachable via SoftAP; NTP unavailable until re-enabled. Scan requires STA mode on.

15.4 ntp_html[] — Time & NTP Page

- Fields: NTP server, GMT offset (seconds), DST offset, auto-sync hours 1–24
- Save → POST /api/ntp
- Sync NTP → POST /api/ntp/sync (requires WiFi)
- Sync Browser → POST /api/time/browser-sync with Math.floor(Date.now()/1000)
- updateTimeStatus shows source (NTP/Browser/RTC/None) + DS3231 presence/age

Default GMT offset in UI load fallback: 28800 (UTC+8), matching initDefaults.

15.5 ap_html[] — Access Point Page

- Edit AP SSID, password (blank=open, else ≥8 chars), channel 1–13, visibility
- Save → POST /api/ap; if restarted flag, toast + reload after 4 s
- Warning alert: changing these disconnects all AP clients

15.6 gpio_html[] — GPIO Configuration Page

JS Function	Behaviour
saveGlobalMode	POST /api/gpio/global-mode {mode:0 1 2}
loadGPIO	GET /api/gpio + global-mode; list pins; fill available select
toggleActiveLow	POST /api/gpio/toggle-active-low {index}
addPin	POST /api/gpio/add {pin}
deletePin	Confirm then POST /api/gpio/delete {index}
resetDefaults	POST /api/gpio/save {pins: DEFAULT_PINS[16]}

Global mode 1/2 disables per-relay polarity buttons. Max 16 relays. Delete shifts indices and warns that schedules renumber.

15.7 system_html[] — System Information Page

Info bar tiles refreshed every 5 s from GET /api/system:

- STA IP, AP IP, Free Heap (KB), Uptime (h m s), WiFi RSSI quality text
- Time Source (colour-coded), UTC Epoch, NTP last sync age, Browser sync age
- NTP Server, Chip Model (ESP32-38P), mDNS hostname.local, Relay count
- GMT offset hours, Drift compensation, GPIO mode text, DS3231 present/sync
- Sync direction label, WiFi Station status
- Verify Services → POST /api/reset (no reboot)
- Factory Reset → confirm → POST /api/factory-reset → redirect home 7 s
- Mentions BOOT button 5 s hardware factory reset alternative

16. Boot Button Factory Reset — checkBootButton()

■ void checkBootButton() — ~lines 2095–2145

Polled every loop iteration. Implements press-hold detector on GPIO0.

```
L0 buttonState = !digitalRead(BOOT_BUTTON_PIN)
  → Active-low with INPUT_PULLUP → true when pressed.
L0 rising edge → bootButtonPressed=true; stamp start
  → Start hold timer.
L0 falling edge → bootButtonPressed=false
  → Release cancels hold.
L0 if held ≥ 5000 ms and not yet triggered → factory path
  → One-shot via factoryResetTriggered guard.
```

Factory path steps (same as web factory reset)

- preferences.clear() entire NVS namespace
- initDefaults(); loadGPIOConfig(); loadConfiguration(); loadExtConfig()
- All relays OFF, clear manual overrides
- If STA enabled with SSID → reconnect
- healer.restartAPIIfNeeded(true); performTargetedRecovery(); updateScheduleCache()
- Clear factoryResetTriggered so another hold can run later

17. GPIO Configuration Persistence

■ void loadGPIOConfig()

```
L0 getBytes("gpioConfig")
  → Read struct from NVS read-only session.
L0 validate len, magic==0xD002, count in 1..16
  → Any failure → install 16 default pins, all activeLow=true, saveGPIOConfig().
```

■ void saveGPIOConfig()

```
L0 putBytes("gpioConfig", &gpioConfig, sizeof)
  → Full struct write.
```

18. NTP Client (64-bit) & Sync

■ void syncInternalRTC(uint64_t rawUtcEpoch)

Central time commit point used by NTP, browser sync, and recovery.

```
L0 reject if !VALID_UNIX_TIME_64
  → Guard garbage epochs.
L0 set internalEpoch, millis/micros anchors, lastRTCRebase, rtcInitialized=true
  → Instant jump of software clock.
L0 if timeSource was BROWSER keep lastBrowserSync else mark NTP + clear browser stamp
  → Preserves browser source when browser path calls this carefully; NTP path overwrites.
L0 ntpFailCount=0; if rtcPresent → immediateDS3231Sync()
  → Mirror good time to hardware.
L0 saveRTCState(); criticalStateDirty=true; updateScheduleCache()
  → Persist + recompute schedules against new clock.
```

■ uint64_t getNTP64Time(const char* server)

Blocking UDP NTP query with up to NTP_RETRY_COUNT (3) attempts. Returns 0 on failure.

Packet construction

```

L0 ntpUDP.begin(2390)
  → Bind local ephemeral-ish port 2390.
L0 packetBuffer[0]=0b11100011
  → LI=0, VN=4, Mode=3 (client).
L0 packetBuffer[1]=0; [2]=6; [3]=0xEC
  → Stratum 0, poll 6, precision -20.
L0 Transmit Timestamp = getCurrentEpoch()+NTP_EPOCH_OFFSET (32-bit seconds)
  → Bytes 40–43 big-endian; fraction bytes 44–47 zeroed.
L0 beginPacket(server,123); write 48; endPacket
  → Send request.

```

Response handling

```

L0 wait up to NTP_TIMEOUT for parsePacket ≥ 48
  → Polling loop with delay(10)+yield.
L0 mode must be 4 (server); LI must not be 3 (unsync)
  → Validate header.
L0 secs = big-endian bytes 40–43; unix = secs - 2208988800
  → Convert NTP era to Unix epoch.
L0 reject if unix < 1577836800 (pre-2020)
  → Sanity floor.
L0 return unixEpoch or 0
  → Success / failure.

```

■ void tryNTPSync()

```

L0 abort if !wifiConnected || !sta_enabled
  → No path to Internet.
L0 throttle by NTP_RETRY_INTERVAL (30 s)
  → Avoid flooding.
L0 query NTP_SERVERS[ntpServerIndex]; on success syncInternalRTC + rotate index
  → Primary attempt.
L0 else walk remaining servers in pool
  → Full failover.
L0 all fail → ++ntpFailCount; ++health.ntpFailures
  → Feed self-healing metrics.

```

19. Browser Time Sync Handler

■ void handleBrowserTimeSync() — POST /api/time/browser-sync

```

L0 require body plain JSON
  → 400 if missing.
L0 parse StaticJsonDocument<256>; read "utc_epoch"
  → Client sends UTC seconds.
L0 validate 1577836800 .. MAX_UNIX_TIME_64
  → 2020 through far future.
L0 timeSource=BROWSER; syncInternalRTC(epoch); force BROWSER again
  → syncInternalRTC may set NTP; handler reasserts BROWSER + lastBrowserSync.
L0 lastNTPSync=0 so NTP can override later when available
  → NTP is preferred when online.
L0 if DS3231 present → rtc.adjust
  → Keep hardware in lockstep.
L0 saveRTCState; updateScheduleCache; criticalStateDirty
  → Persist + reschedule.
L0 respond JSON: success, utc_epoch, local_time, gmt_offset, time_source, rtc_* , drift
  → UI updates clock immediately from local_time.

```

20. WiFi Connection Logic

■ void beginWiFiConnect()

```

L0 guards: sta_enabled, non-empty SSID, past giveUpUntil, not already connecting, not paused
  → All must pass.
L0 WiFi.disconnect(true); delay(100); ++wifiReconnectAttempts
  → Clean slate each attempt.
L0 ensure WIFI_AP_STA; WiFi.begin(ssid, password)
  → Start association; set wifiConnecting + wifiConnectStart.

```

Connection state machine (inside loop)

- While wifiConnecting: poll WiFi.status(); also handleClient+yield so UI stays alive
- WL_CONNECTED → clear flags, reset attempts, tryNTPSync immediately
- Timeout 20 s → stop attempt; if attempts≥10 and not firstAttempt → giveUpUntil = now+5 min
- Every WIFI_CHECK_INTERVAL (10 s): detect disconnect, schedule beginWiFiConnect
- wifiPausedForScan auto-clears at wifiPauseUntil or 30 s after lastScanAttempt

21. Relay Output Engine

■ void updateRelayOutputs()

Bulk apply: for each active relay, state = manualOverride ? manualState : scheduleActiveCache[i]. Only calls setRelayOutput when state changed or first init. Sets relayOutputsInitialized.

Note: the primary live path in loop is processRelaySchedules(), which drives pins directly with debounce. updateRelayOutputs remains available for bulk reconcile.

22. Schedule Cache & Schedule Engine

■ void updateScheduleCache()

Rebuilds scheduleActiveCache[] from current local time. Does not drive pins.

```

L0 epoch=getCurrentEpoch(); reject if < MIN_UNIX_TIME_64
  → Need valid clock.
L0 localEpoch → gmtime64 → cache todayBit, monthDay, month
  → Calendar context for all relays.
L0 cur = hour*3600+min*60+sec; yesterdayBit = 1<&&&((wday+6)%7)
  → Seconds-of-day + previous weekday for overnight.
L0 for each relay: if manualOverride → cache false (manual handled elsewhere)
  → Cache only represents schedule desire.
L0 for each of 8 slots if enabled:
  → Evaluate filters then time window.
    • Month filter: if monthMask non-zero and bit for current month clear → skip
    • DOM filter: if monthDays non-zero and bit for (mday-1) clear → skip
    • start==stop → Always ON if today in days mask
    • start<stop → same-day window [start, stop) if today matches
    • start>stop → overnight: ON if (cur≥start & today) OR (cur<stop & yesterday)
L0 scheduleActiveCache[i]=hasActive; stamp lastScheduleCacheUpdate
  → OR across slots — any matching slot turns channel ON.

```

■ void processRelaySchedules()

Called every SCHEDULE_PROCESS_INTERVAL (250 ms). Refreshes cache every 1 s. Drives pins with 500 ms debounce on schedule-driven changes.

```

L0 refresh updateScheduleCache if cache interval elapsed
  → Keep calendar fields fresh.
L0 recompute shouldBeOn with same rules as cache (inline duplicate logic)
  → Uses cached todayBit/monthDay/month for speed.
L0 manualOverride path: immediately set pin if differs; mark criticalStateDirty
  → No debounce on manual — instant UI response.
L0 schedule path: if shouldBeOn != lastDebouncedState and 500 ms since last change → drive
  → Prevents chatter near boundaries / RTC jumps.
L0 relayOutputsInitialized = true
  → Mark outputs known.

```

■ void restartAP()

Thin wrapper: healer.restartAPIfNeeded(false) — non-forced path.

23. mDNS Functions

■ void startMDNS()

L0 return if already started
→ Idempotent.

L0 if hostname empty or default → derive from ap_ssid
→ lowercase, spaces/underscores → hyphens, strip non-alnum, max 31 chars.

L0 MDNS.begin(hostname); addService http/tcp/80; TXT model/version/channels/features
→ Advertises http://<host>.local/

■ void stopMDNS()

L0 if started: MDNS.end(); mdnsStarted=false
→ Tear down responder.

■ void restartMDNS()

Delegates to healer.liveReconfigureMDNS().

■ void scheduleMDNSRestart()

L0 mdnsRestartScheduled=true; pending=millis()+2000
→ Deferred restart so HTTP response can finish.

■ String getMDNSHostname()

Returns String(mdnsHostname).

■ void setMDNSHostname(const char* hostname)

L0 validate length 1..31; sanitise to [a-z0-9-]
→ Upper→lower; space/_ → '-.

L0 if running → scheduleMDNSRestart()
→ Apply after delay.

24. setup() — Full Boot Sequence

Executed once at power-on / reset. Ordered carefully so relays start safe (OFF) before networking.

#	Call / Action	Purpose
1	initRTC()	Probe DS3231; may seed software clock from hardware
2	loadGPIOConfig()	Restore pin map / polarity from NVS or defaults
3	pinMode(BOOT, INPUT_PULLUP)	Enable factory-reset button
4	for each relay: pinMode OUTPUT; setRelayOutput(false)	Safe de-energised boot
5	initScheduleDefaults(all 16)	RAM defaults before NVS overlay
6	loadConfiguration(); loadExtConfig()	SSID, schedules, AP, NTP, offsets, flags
7	Time priority: loadRTCFromDS3231() else loadRTCState() else NONE	Best available clock
8	WiFi.mode AP_STA or AP	Depends on sta_enabled
9	If STA enabled + SSID → WiFi.begin	Start background connect
10	Clamp AP channel; softAPdisconnect; softAP(ssid, pass, ch, hidden)	Bring up configuration AP
11	startMDNS()	Local name discovery
12	dnsServer.start(53, "*", softAPIP)	Captive portal DNS wildcard
13	setupWebServer()	Register all routes + server.begin()

#	Call / Action	Purpose
14	updateScheduleCache()	Initial schedule evaluation
15	healer.restoreCriticalState()	Re-apply manual overrides if checksum OK
16	Init memory/connection timestamps	Baseline for loop maintenance

25. loop() — Main Runtime Loop

Non-blocking cooperative loop. Order matters: button and HTTP first for responsiveness, then maintenance, time, WiFi, NTP, schedules, and critical-state flush.

Phase	Interval / Trigger	Actions
Input	every iter	checkBootButton()
HTTP/DNS	every iter	server.handleClient(); dnsServer.processNextRequest()
Activity	every iter	if client → lastConnectionActivity=now
Conn cleanup	60 s	stop up to 5 leftover clients; scanDelete if idle
Mem clean	MEMORY_CLEANUP_INTERVAL 30 s	cleanupStaleResources()
Web health	60 s	checkWebServerHealth()
Self-heal	10 s	healer.smartRecovery()
Heap	via checkAndCleanMemory	low-heap + hourly cleanup
DS3231 pull	DS3231_SYNC_INTERVAL 1 h	If present+valid: internalEpoch←rtc.now(); source=RTC unless NTP
RTC autosave	hourly when offline	autoSaveInternalRTC()
mDNS rename	deadline	restartMDNS if scheduled
Scan timeout	10 s	abort stuck scans
STA connect FSM	while connecting	success→NTP; timeout→backoff
Scan pause end	deadline / 30 s	clear wifiPausedForScan
WiFi check	10 s	reconnect logic + detect silent connect
NTP	interval / fail retry	tryNTPSync when connected
Schedules	250 ms	processRelaySchedules()
Critical NVS	5 min if dirty	healer.saveCriticalState()
Yield	end	yield() to WiFi stack

26. Configuration Load/Save (NVS)

■ void initDefaults()

```
L0 memset sysConfig; magic=0x1234; version=11
  → Clean slate header.
L0 ap_ssid="ESP32_16CH_Timer_Switch"; ap_password="12345678"
  → Default SoftAP credentials (openable by changing password to blank later).
L0 ntp_server="time.google.com"; gmt_offset=28800; daylight=0
  → UTC+8 default timezone.
L0 last_rtc_epoch=0; rtc_drift=1.0; hostname="esp32"
  → Clock empty until sync.
L0 initScheduleDefaults all relays; default 16 GPIO activeLow
  → Full factory pin map.
L0 timeSource=NONE; saveConfiguration+saveGPIOConfig
  → Persist immediately.
```

■ void loadConfiguration()

```
L0 getBytes("sysConfig") – if bad magic/len → initDefaults and reload
  → Self-healing config.
L0 if version < 11: if <10 fill monthMask=MONTH_ALL; set version 11; save
  → Migration path for month-of-year feature.
L0 getBytes("relayConfigs") – if size mismatch re-init all schedules
  → Protect against struct size changes.
```

L0 copy ap_ssid/ap_password into runtime char buffers
→ Used by some legacy paths.

■ void saveConfiguration()

L0 putBytes sysConfig + putBytes relayConfigs
→ Atomic-ish pair in one RW session.

■ void loadExtConfig()

L0 if bad magic/len → defaults: ch=6, syncHours=1, hidden=0, globalMode=0, sta=1
→ Then saveExtConfig.
L0 else clamp channel 1-13, hours 1-24, globalMode≤2, sta_enabled≤1
→ Sanitise corrupted fields.

■ void saveExtConfig()

L0 putBytes("extConfig")
→ Persist extended flags.

27. Web Server Route Table — setupWebServer()

Registers every HTTP route then `server.begin()`. Content pages use `send_P` (PROGMEM).

27.1 HTML / CSS Pages

Method	Path	Response
GET	/	index_html (Relays UI)
GET	/wifi	wifi_html
GET	/ntp	ntp_html
GET	/ap	ap_html
GET	/gpio	gpio_html
GET	/system	system_html
GET	/style.css	style_css

27.2 REST JSON API

Method	Path	Handler
GET	/api/relays	handleGetRelays
POST	/api/relay/manual	handleManualControl
POST	/api/relay/reset	handleResetManual
POST	/api/relay/save	handleSaveRelay
POST	/api/relay/name	handleRelayName
GET	/api/time	handleGetTime
POST	/api/time/browser-sync	handleBrowserTimeSync
GET	/api/wifi	handleGetWiFi
POST	/api/wifi	handleSaveWiFi
POST	/api/wifi/scan	handleWiFiScanStart
GET	/api/wifi/scan	handleWiFiScanResults
GET	/api/ntp	handleGetNTP
POST	/api/ntp	handleSaveNTP
POST	/api/ntp/sync	handleSyncNTP
GET	/api/ap	handleGetAP
POST	/api/ap	handleSaveAP
GET	/api/gpio	handleGetGPIOConfig
POST	/api/gpio/save	handleSaveGPIOConfig
POST	/api/gpio/add	handleAddGPIO
POST	/api/gpio/delete	handleDeleteGPIO
POST	/api/gpio/toggle-active-low	handleToggleActiveLow
GET	/api/gpio/global-mode	inline $\lambda \rightarrow \{\text{mode}\}$
POST	/api/gpio/global-mode	handleGlobalActiveMode
GET	/api/mdns	inline $\rightarrow \text{hostname/started/url}$
POST	/api/mdns	inline <code>setMDNSHostname</code>
POST	/api/mdns/restart	inline <code>restartMDNS</code>
GET	/api/system	handleGetSystem
POST	/api/reset	handleReset

Method	Path	Handler
POST	/api/factory-reset	handleFactoryReset

27.3 Captive Portal Compatibility Routes

Path	Behaviour
/hotspot-detect.html	iOS captive success HTML
/library/test/success.html	iOS variant success HTML
/generate_204	Android → 302 to SoftAP IP /
/success.txt	plain "success"
/canonical.html	302 to SoftAP /
/connecttest.txt	Microsoft Connect Test body
/ncsi.txt	Microsoft NCSI body
/redirect	302 to SoftAP /
onNotFound	any other path → 302 to SoftAP /

28. REST API Handlers (Every Endpoint, Full Detail)

■ void handleGetRelays() — GET /api/relays

Returns a JSON array of relay objects. Uses chunked transfer (CONTENT_LENGTH_UNKNOWN) to stream large payloads without one giant String.

```
L0 touch lastConnectionActivity
  → Keep web health happy.
L0 if responseCache valid &lt;2 s and non-empty → send cached (currently cache often invalidated)
  → Fast path when enabled.
L0 send 200 + open '[' ; for each active relay serialize StaticJsonDocument&lt;512&gt;;
  → Per-relay object then sendContent; yield between relays.
```

Each element fields:

- state (bool) — lastRelayOutputs[i]
- manual (bool) — manualOverride
- name (string) — relay name
- pin (int) — GPIO number
- schedules[8]: startHour/Minute/Second, stopHour/Minute/Second, enabled, days, monthDays, monthMask

```
L0 close ']'; responseCache.valid=false
  → End stream.
```

■ void handleManualControl() — POST /api/relay/manual

Body: {"relay": <int>, "state": <bool>}

```
L0 parse JSON; bounds-check relay index
  → 400 on bad input.
L0 manualOverride=true; manualState=state; scheduleActiveCache=false
  → Freeze schedule influence.
L0 setRelayOutput; lastRelayOutputs=state; saveConfiguration
  → Immediate pin drive + persist.
L0 criticalStateDirty=true; cache invalid; {success:true}
  → Checkpoint soon.
```

■ void handleResetManual() — POST /api/relay/reset

Body: {"relay": <int>} — clears manualOverride, updateScheduleCache, save, {success:true}.

■ void handleSaveRelay() — POST /api/relay/save

Body: {"relay": n, "schedules": [up to 8 objects]}

```
L0 StaticJsonDocument&lt;5120&gt;;
  → Large enough for 8 full schedule objects.
L0 copy start/stop hms, enabled, days, monthDays, monthMask|MONTH_ALL
  → monthMask defaults to all months if omitted.
L0 saveConfiguration; updateScheduleCache; invalidate cache; success
  → Live schedules take effect within ≤1 s cache refresh / 250 ms process.
```

■ void handleRelayName() — POST /api/relay/name

Body: {"relay": n, "name": "..."} — strncpy 15 chars + NUL; save; success.

■ void handleGetTime() — GET /api/time

Compact status used by every page's 1 Hz tick().

- time — local HH:MM:SS or --:--:--
- wifi — bool connected
- ntp — bool (timeSource==NTP)
- timeSource — "none"|"ntp"|"browser"|"rtc"
- rtcPresent, rtcSynced, rtcSyncAge (seconds)

■ void handleGetWiFi() — GET /api/wifi

JSON: ssid, connected, ip, rssi, sta_enabled.

■ void handleSaveWiFi() — POST /api/wifi

```
L0 if body has "sta_enabled" → setWiFiStationEnabled; return early
  → Toggle path separate from credentials.
L0 else require ssid 1..31 chars; optional password
  → Empty password clears stored PSK.
L0 if STA enabled and creds changed → disconnect, delay 500, begin reconnect FSM
  → Immediate association attempt.
```

■ void handleWiFiScanStart() — POST /api/wifi/scan

```
L0 400 if !sta_enabled
  → Scan needs station radio path.
L0 if connecting → pauseWiFiForScan()
  → Free radio.
L0 if !scanInProgress → async WiFi.scanNetworks(true,true); 202 {scanning:true}
  → show_hidden=true second arg.
```

■ void handleWiFiScanResults() — GET /api/wifi/scan

```
L0 if still WIFI_SCAN_RUNNING → {scanning:true}
  → UI keeps polling.
L0 else build networks[{ssid,rssi,enc}] up to 30; scanDelete; clear count
  → enc true if not OPEN.
L0 if was paused → extend wifiPauseUntil +5 s
  → Grace before reconnect.
```

■ void handleGetNTP() — GET /api/ntp

ntpServer, gmtOffset, daylightOffset, syncHours, globalActiveMode (note: polarity also echoed here).

■ void handleSaveNTP() — POST /api/ntp

Updates ntp_server, gmt_offset, daylight_offset; if syncHours 1–24 saves extConfig too.

■ void handleSyncNTP() — POST /api/ntp/sync

```
L0 400 if WiFi down or STA disabled
  → Clear error for UI.
L0 reset ntpAsyncState/lastNTPSync/lastNTPAttempt → loop will tryNTPSync soon
  → Non-blocking initiate; returns success message immediately.
```

■ void handleGetAP() — GET /api/ap

ap_ssid, ap_password, ap_channel, ap_hidden. Note: password is returned in JSON (local trusted network assumption).

■ void handleSaveAP() — POST /api/ap

```
L0 update ssid if 1..31; password if ≥8 or blank to open
  → Track change flags.
L0 channel 1-13; hidden bool
  → ExtConfig fields.
L0 save both configs; if any change → restartAPIfNeeded(true); {restarted:true}
  → Clients must reconnect.
```

■ void handleGetSystem() — GET /api/system

Rich diagnostics object (~2 KB DynamicJsonDocument):

- ip, ap_ip, uptime (s), freeHeap, utcEpoch (stringified uint64)
- timeSource (None/NTP/Browser/RTC), ntpServer
- ntpSyncAge / browserSyncAge / rtcSyncAge (0xFFFFFFFF = never)
- wifiConnected, wifiSSID, rssi, version, chipModel=ESP32-38P
- mdnsHostname, mdnsStarted, relayCount, maxRelays
- gmtOffset, driftComp, globalActiveMode, rtcPresent, staEnabled

■ `void handleReset()` — POST `/api/reset`

Responds success then `healer.performTargetedRecovery()` — live service verification without reboot or config wipe.

■ `void handleFactoryReset()` — POST `/api/factory-reset`

Identical logical steps to BOOT button path: clear NVS, `initDefaults`, reload, relays OFF, optional STA reconnect, AP restart, targeted recovery, schedule cache refresh, JSON success message. Does **not** call `ESP.restart()` — device keeps running on defaults.

29. GPIO Management Handlers

■ void handleGetGPIOConfig() — GET /api/gpio

- count, maxRelays, pins[], activeLow[], availablePins[]
- availablePins is the fixed valid set matching DEFAULT_RELAY_PINS (not “free only” — UI filters used)

■ void handleSaveGPIOConfig() — POST /api/gpio/save

Body: {"pins": [gpio, ...]} length ≤ 16. Used by Reset to Default.

```
L0 snapshot old manual/name/schedule/activeLow
  → Preserve per-index where possible.
L0 assign new pins; restore old meta for overlapping indices
  → Extra slots get initScheduleDefaults + activeLow true.
L0 pinMode all; force OFF; save gpio+config; update cache; dirty critical
  → Immediate safe apply.
```

■ void handleAddGPIO() — POST /api/gpio/add

Body: {"pin": n}

```
L0 reject if count≥16 or pin already used
  → 400 errors.
L0 append pin, activeLow=true, initScheduleDefaults, count++
  → pinMode OUTPUT; drive OFF; save both; update cache.
```

■ void handleDeleteGPIO() — POST /api/gpio/delete

Body: {"index": i}

```
L0 setRelayOutput false; shift pins/activeLow/relayConfigs/lastRelayOutputs down
  → Compact array; re-init vacated last slot; count--.
```

Side effect: relay numbers after the deleted index shift down by one; schedules move with the slot.

■ void handleToggleActiveLow() — POST /api/gpio/toggle-active-low

Body: {"index": i} — flips activeLow[i], saveGPIO, re-init pin OFF, return new activeLow bool. UI should disable this when global mode ≠ 0 (enforced in JS, not server).

■ void handleGlobalActiveMode() — POST /api/gpio/global-mode

Body: {"mode": 0|1|2}

```
L0 validate mode≤2; extConfig.global_active_mode=mode; saveExtConfig
  → Persist.
L0 re-drive every relay with setRelayOutput(current logical state)
  → Applies new polarity without changing logical ON/OFF.
L0 critical dirty; cache invalid; {success, mode}
  → Done.
```

30. Captive Portal Behaviour

dnsServer is started in setup with wildcard "*" redirecting all DNS queries to the SoftAP IP. Combined with OS-specific probe URLs (section 27.3) and onNotFound → 302 to AP root, phones and laptops typically pop a captive login browser pointing at the Relays UI. No authentication is implemented on the web UI — physical proximity / WiFi password is the trust boundary.

31. Runtime Architecture & Data Flow

31.1 Time Authority Priority

Boot:

```
DS3231 valid? → seed software RTC (TIME_SOURCE_RTC)
else NVS last_rtc_epoch valid? → seed software RTC
else TIME_SOURCE_NONE (schedules idle until sync)
```

Runtime upgrades:

```
Browser sync → TIME_SOURCE_BROWSER (also writes DS3231)
NTP success → TIME_SOURCE_NTP (preferred; clears browser stamp)
Hourly DS3231 read → may set TIME_SOURCE_RTC if not on NTP
```

getCurrentEpoch() always prefers software clock extrapolation;
falls back to live DS3231 read only if software not initialised.

31.2 Relay Decision Path

```
every 250 ms processRelaySchedules():
  if manualOverride[i]:
    drive pin ← manualState[i] (immediate)
  else:
    shouldBeOn ← OR over 8 schedules matching:
      enabled &&& monthMask &&& monthDays &&& days &&& time-window
    debounce 500 ms then drive pin ← shouldBeOn
```

```
setRelayOutput(i, logical):
  gpio ← pins[i]
  level ← isActiveLow(i) ? !logical : logical
  digitalWrite(gpio, level)
```

31.3 NVS Key Map

Key	Type / Size	Writer
sysConfig	SystemConfig packed	saveConfiguration / saveRTCState
relayConfigs	RelayConfig × 16	saveConfiguration
extConfig	ExtConfig packed	saveExtConfig
gpioConfig	GPIOPinConfig	saveGPIOConfig
criticalState	CriticalRelayState	saveCriticalState

Namespace: "relay16". preferences.clear() wipes all keys (factory reset).

31.4 Network Modes

sta_enabled	WiFi.mode	Internet/NTP	Config access
1 (default)	WIFI_AP_STA	Yes if joined	STA IP and/or AP IP + mDNS
0	WIFI_AP	No	AP IP + captive portal + mDNS

32. Complete Feature Checklist (Verification Matrix)

Use this list to confirm documentation coverage against the binary feature set:

Scheduling

- 8 schedules × 16 relays
- Second-resolution start/stop
- DOW bitmask

- DOM bitmask (31 days)
- MOY bitmask (12 months)
- Overnight windows
- Always-ON (start==stop)
- 500 ms debounce
- 250 ms process tick
- 1 s cache refresh

Manual control

- Force ON
- Force OFF
- Return to Auto
- Persisted in NVS
- Restored via criticalState

GPIO

- Dynamic count 0–16
- Add pin
- Delete pin (shift)
- Reset default map
- Per-relay active level
- Global active LOW
- Global active HIGH
- Immediate re-drive on polarity change

Timekeeping

- 64-bit epoch
- gmtime64/localtime64
- Software RTC + micros drift
- 5 min rebase
- Hourly offline autosave
- DS3231 I2C 21/22
- lostPower detection
- NTP 4-server pool
- NTP interval 1–24 h
- Browser sync API
- Timezone + DST offsets
- Source indicator in UI

WiFi

- SoftAP always
- STA optional
- STA enable toggle
- Scan async
- Scan pause
- Reconnect backoff
- RSSI reporting
- Channel + hidden AP
- Captive portal DNS

Services

- WebServer :80
- Full REST API
- PROGMEM UI
- mDNS + TXT
- Self-healing
- Relay pin verify
- Memory cleanup
- Factory reset web
- Factory reset BOOT 5 s
- Service verify without reboot

UI pages

- Relays
- WiFi
- Time
- AP
- GPIO
- System
- Shared CSS
- Toasts
- Live clock header

33. Default Values Reference

Item	Default
AP SSID	ESP32_16CH_Timer_Switch
AP Password	12345678
AP Channel	6
AP Hidden	false (0)
STA Enabled	true (1)
STA SSID/Password	empty
NTP Server	time.google.com
GMT Offset	28800 s (UTC+8)
Daylight Offset	0
NTP Sync Hours	1
Hostname seed	esp32
Relay count	16
All activeLow	true
Global active mode	0 (per-relay)
Schedule days	DAY_ALL (0x7F)
Schedule monthDays	0 (any)
Schedule monthMask	MONTH_ALL (0x0FFF)
Schedule enabled	false
Manual override	false
Relay names	Relay 1 ... Relay 16
Drift compensation	1.0
Config magic	0x1234
Config version	11
Ext magic	0xEC
GPIO magic	0xD002
Critical magic	0xDEADBEEF

34. Error Handling & Edge Cases

Edge Case	Handling
Millis wrap (~49.7 days)	All intervals use unsigned subtraction via <code>timeHasElapsed</code> / <code>isTimeReached</code> .
Micros wrap (~71.6 min)	<code>performRTCreabase</code> and <code>getCurrentEpoch</code> detect <code>current < anchor</code> and recompute.
Invalid NVS blob	Magic/size fail → <code>initDefaults</code> or field clamps; never crashes on corrupt flash.
DS3231 missing	<code>rtcPresent=false</code> ; system runs on NTP/browser/software clock only.
DS3231 lostPower	Time not trusted until external sync writes <code>adjust()</code> .
No time yet	<code>getCurrentEpoch=0</code> ; <code>updateScheduleCache</code> returns early; relays stay off unless manual.
NTP all servers fail	<code>ntpFailCount/health.ntpFailures</code> increment; retry every 30 s while online.
WiFi give-up	After 10 failed reconnects, pause 5 minutes (<code>wifiGiveUpUntil</code>).
Scan during connect	<code>pauseWiFiForScan</code> disconnects STA 15 s to allow clean scan.
Low heap <20 KB	<code>performMemoryCleanup</code> triggered from <code>checkAndCleanMemory</code> .

Edge Case	Handling
Relay pin mismatch	verifyRelayStates re-writes expected level every 30 s.
AP IP 0.0.0.0	liveReconfigureAP / smartRecovery recreates SoftAP.
Overnight schedule	Uses yesterday DOW bit for the post-midnight tail.
monthDays=0	Means any day of month (filter disabled).
monthMask=0	Treated like all months in engine (if monthMask non-zero and bit clear, skip).
Factory reset no reboot	Services reconfigured live; UI should reload.
GPIO 1/3 as relays	Conflicts with UART0 serial; documented hardware caveat.
JSON too large	handleSaveRelay uses 5120-byte doc; oversized body yields Bad JSON 400.
Name length	Server caps at 15 chars; UI maxLength=15.
AP password <8	UI rejects; server ignores short non-empty passwords (keeps old).

35. Hardware Pinout & Wiring Notes

35.1 I2C DS3231

- SDA → GPIO 21
- SCL → GPIO 22
- VCC 3.3 V, GND common; backup battery recommended for lostPower=false across outages

35.2 BOOT Button

- GPIO 0 — hold LOW 5 s for factory reset (built-in BOOT button on most DevKit boards)

35.3 Default Relay Driver Inputs

Map logical channels to a 16-channel active-low relay module's INx pins using the table in §3.13. Most opto-isolated modules are active-LOW (default polarity). If your board is active-HIGH, set Global Mode 2 or toggle per channel on the GPIO page.

Ensure relay module VCC/JD-VCC power budget is external when switching mains loads. ESP32 GPIO only provides signal level, not coil current for large banks.

35.4 Power & Brownout

Critical relay state is checkpointed to NVS when dirty (at most every 5 minutes, and on full health check). After unexpected reset, manual overrides are restored; schedule-driven states recompute from clock. Keep a DS3231 with battery so schedules resume correctly without waiting for WiFi/NTP.

Appendix A — Complete Function Index

Every free function and class method defined in the sketch:

Function / Method	Summary
timeHasElapsed	Millis interval check
isTimeReached	Millis deadline check
rtcDateTimeToUint64	DateTime → uint64 epoch
uint64ToRtcDateTime	uint64 → DateTime
gmtime64	UTC broken-down time (64-bit)
localtime64	Local broken-down time
getLocalEpoch	UTC→local epoch seconds
isActiveLow	Resolve polarity with global mode
getNTPInterval	Hours→ms NTP period
getRelayPin	Index→GPIO or -1
getActiveRelayCount	gpioConfig.count
setRelayOutput	Drive pin with polarity
initScheduleDefaults	Factory RelayConfig slot
setWiFiStationEnabled	Enable/disable STA live
initRTC	Probe DS3231 at boot
immediateDS3231Sync	Push epoch to DS3231
syncDS3231FromInternalRTC	Push without begin()
performRTCReabase	Absorb micros into epoch
getCurrentEpoch	Live UTC epoch read
saveRTCState	Persist epoch to NVS
loadRTCState	Restore epoch from NVS
autoSaveInternalRTC	Hourly offline save
loadRTCFromDS3231	Pull DS3231→software
pauseWiFiForScan	Temp STA disconnect
calculateCriticalChecksum	Checksum blob
SelfHealingSystem::saveCriticalState	NVS checkpoint
SelfHealingSystem::restoreCriticalState	Boot restore manuals
SelfHealingSystem::liveReconfigureWiFi	Soft WiFi fix
SelfHealingSystem::liveReconfigureMDNS	mDNS announce
SelfHealingSystem::liveReconfigureDNS	DNS stub
SelfHealingSystem::liveReconfigureWebServer	Web stub
SelfHealingSystem::liveReconfigureAP	AP if down
SelfHealingSystem::restartAPIfNeeded	Forced AP bounce
SelfHealingSystem::recoverWiFi	WiFi recovery
SelfHealingSystem::recoverMDNS	mDNS recovery
SelfHealingSystem::recoverDNS	DNS recovery
SelfHealingSystem::recoverWebServer	Web recovery
SelfHealingSystem::recoverNTP	Multi-server NTP
SelfHealingSystem::recoverRTC	I2C RTC recover

Function / Method	Summary
SelfHealingSystem::performTargetedRecovery	Full ordered recover
SelfHealingSystem::verifyRelayStates	Pin level audit
SelfHealingSystem::smartRecovery	10 s health tick
performMemoryCleanup	Heap/scan hygiene
cleanupStateResources	Cache+scan cleanup
checkWebServerHealth	Idle web touch
checkAndCleanMemory	Heap monitor
checkBootButton	5 s factory reset
loadGPIOConfig	NVS→gpioConfig
saveGPIOConfig	gpioConfig→NVS
syncInternalRTC	Commit new epoch
getNTP64Time	UDP NTP query
tryNTPSync	Pool failover sync
handleBrowserTimeSync	API browser time
beginWiFiConnect	Start STA attempt
updateRelayOutputs	Bulk pin apply
updateScheduleCache	Recompute schedule flags
processRelaySchedules	Engine + debounce drive
restartAP	Wrapper AP restart
startMDNS	Begin responder
stopMDNS	End responder
restartMDNS	Live reconfigure
scheduleMDNSRestart	Deferred restart
getMDNSHostname	Read hostname
setMDNSHostname	Sanitise+set hostname
setup	Boot sequence
loop	Main runtime
initDefaults	Factory settings
loadConfiguration	Load sys+relays
saveConfiguration	Save sys+relays
loadExtConfig	Load extended
saveExtConfig	Save extended
setupWebServer	Route table + begin
handleGetRelays	GET relays JSON
handleManualControl	POST manual
handleResetManual	POST auto
handleSaveRelay	POST schedules
handleRelayName	POST name
handleGetTime	GET time status
handleGetWiFi	GET wifi status
handleSaveWiFi	POST wifi/sta
handleWiFiScanStart	POST scan
handleWiFiScanResults	GET scan results
handleGetNTP	GET ntp settings

Function / Method	Summary
handleSaveNTP	POST ntp settings
handleSyncNTP	POST ntp sync
handleGetAP	GET ap settings
handleSaveAP	POST ap settings
handleGetSystem	GET system info
handleReset	POST verify services
handleFactoryReset	POST factory reset
handleGetGPIOConfig	GET gpio map
handleSaveGPIOConfig	POST pins array
handleAddGPIO	POST add pin
handleDeleteGPIO	POST delete index
handleToggleActiveLow	POST flip polarity
handleGlobalActiveMode	POST global mode

End of documentation. This reference covers every section, structure, constant, function, API route, UI page, and feature present in esp32-final-sketch.ino (Copy) without intentional omissions. Source author: Raff Alds · License: GPLv3 · Config v11.

Generated as a complete technical manual for maintenance, auditing, and onboarding.